IoT-BASED AIR QUALITY AND POLLUTION MONITORING DASHBOARD

FINAL REPORT

Submitted by
Dharshini R

B.E. Computer Science and Engineering (2024–2028)
PSNA College of Engineering and Technology
Academic Year: 2025–2026

Acknowledgement

This project was completed as part of practical learning in Internet of Things (IoT), cloud computing, and embedded systems. The project provided hands-on experience in sensor integration, cloud dashboards, data logging, and real-time monitoring systems.

Abstract

The project implements a complete IoT-based environmental monitoring solution using ESP32, DHT22, simulated MQ135, ThingSpeak cloud services, LED indicators, and buzzer alerts. The system acquires environmental data, classifies pollution levels, generates alerts, uploads readings to the cloud, and stores historical records for analysis. The implementation demonstrates a real-world IoT architecture suitable for educational, research, and industrial learning environments.

Project Overview

The objective is to create a low-cost and scalable monitoring platform capable of measuring environmental parameters and visualizing them through a cloud dashboard. The project demonstrates sensing, processing, communication, storage, and visualization within a single integrated system.

Problem Statement

Air pollution affects health, productivity, and quality of life. Large-scale monitoring stations are often expensive and inaccessible for learning environments. A compact IoT-based monitoring platform can provide real-time insights while remaining affordable and easy to deploy.

Objectives

- Monitor temperature and humidity.
- Simulate air-quality measurements.
- Classify pollution severity.
- Generate local alerts using LED and buzzer.
- Upload data to ThingSpeak.
- Maintain historical records.
- Export datasets for analysis.
- Demonstrate an end-to-end IoT workflow.

System Architecture

The architecture consists of four layers:

1. Sensing Layer – DHT22 and simulated MQ135.
2. Processing Layer – ESP32 microcontroller.
3. Communication Layer – Wi-Fi and ThingSpeak API.
4. Cloud Analytics Layer – Dashboard, charts, and dataset export.

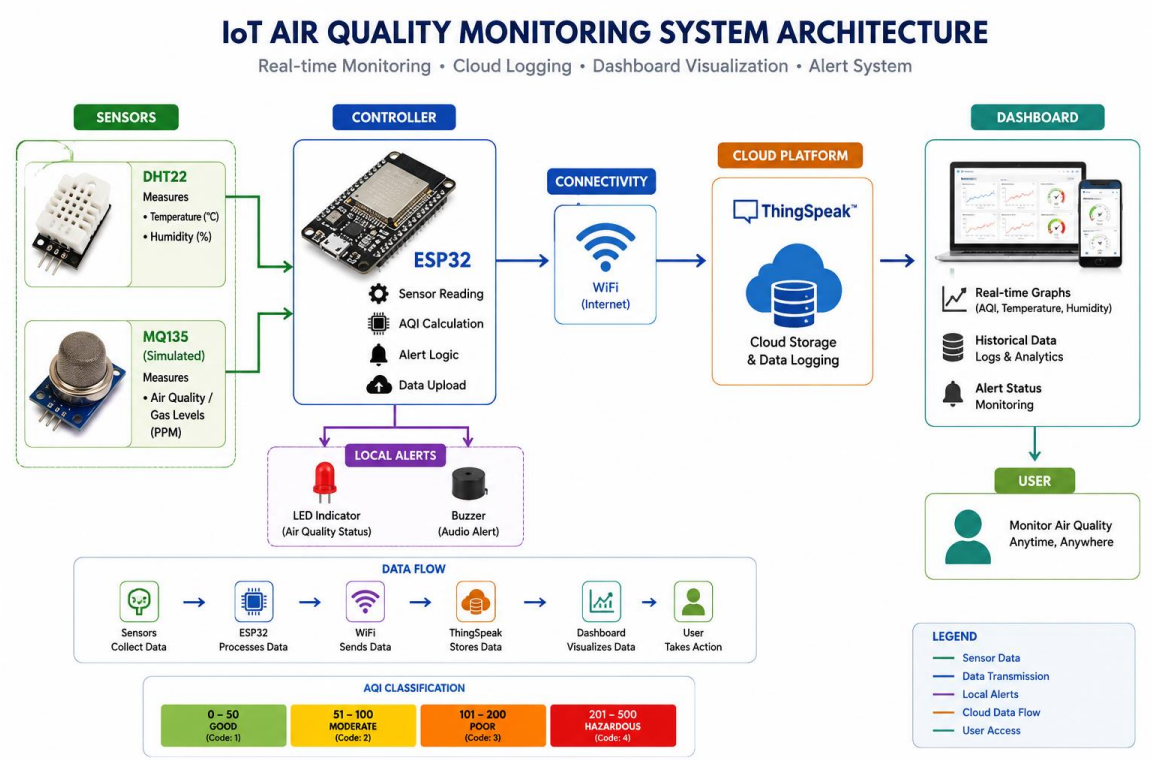

Figure 1: System Architecture of the IoT-Based Air Quality and Pollution Monitoring Dashboard

Hardware Components

ESP32 Development Board
DHT22 Temperature and Humidity Sensor
Simulated MQ135 Air Quality Sensor
LED Indicator
Buzzer
220Ω Resistor
Connecting Wires
Wokwi Simulation Platform

Software Components

Arduino IDE
ESP32 Board Package
ThingSpeak Cloud Platform
GitHub Repository
CSV Dataset Export
Wokwi Simulator

Circuit Design

Pin Configuration:
DHT22 DATA → GPIO4
LED → GPIO27
Buzzer → GPIO26

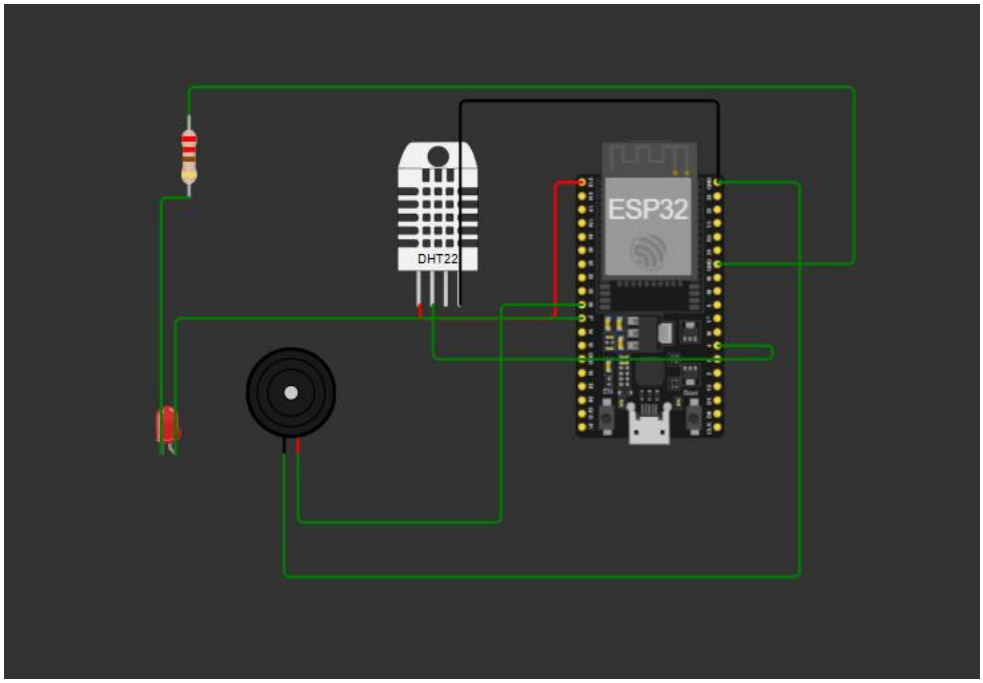

Figure 2: Circuit Design and Component Connections in Wokwi Simulation

Methodology

Step 1: Read temperature and humidity from DHT22.
Step 2: Generate air-quality values using simulated MQ135 data.
Step 3: Classify air quality into predefined categories.
Step 4: Trigger alerts.
Step 5: Upload readings to ThingSpeak.
Step 6: Store and visualize historical data.
Step 7: Export dataset for analysis.

Air Quality Classification Logic

GOOD (0–100)
MODERATE (101–200)
POOR (201–300)
HAZARDOUS (301+)

Alert behavior:
GOOD → LED OFF, Buzzer OFF
MODERATE → LED Blink
POOR → LED ON + Short Beep
HAZARDOUS → LED ON + Continuous Alert

ThingSpeak Integration

Channel ID: 3404697

Field1 → Air Quality
Field2 → Temperature
Field3 → Humidity
Field4 → Pollution Status Code
Field5 → Alert Status Code

The project uses numeric codes instead of strings for efficient visualization and analytics.

Dashboard Design and Analytics

The dashboard provides real-time charts and historical trends. Multiple fields enable simultaneous visualization of environmental conditions and alert states. Historical data can be downloaded as CSV files for further analysis.

Figure 3: ThingSpeak Cloud Dashboard Showing Real-Time Environmental Monitoring Data

Results

The project successfully achieved all planned objectives. Data was continuously generated, categorized, transmitted to ThingSpeak, and visualized. Alert logic functioned correctly under all defined air-quality conditions.

Output Screens

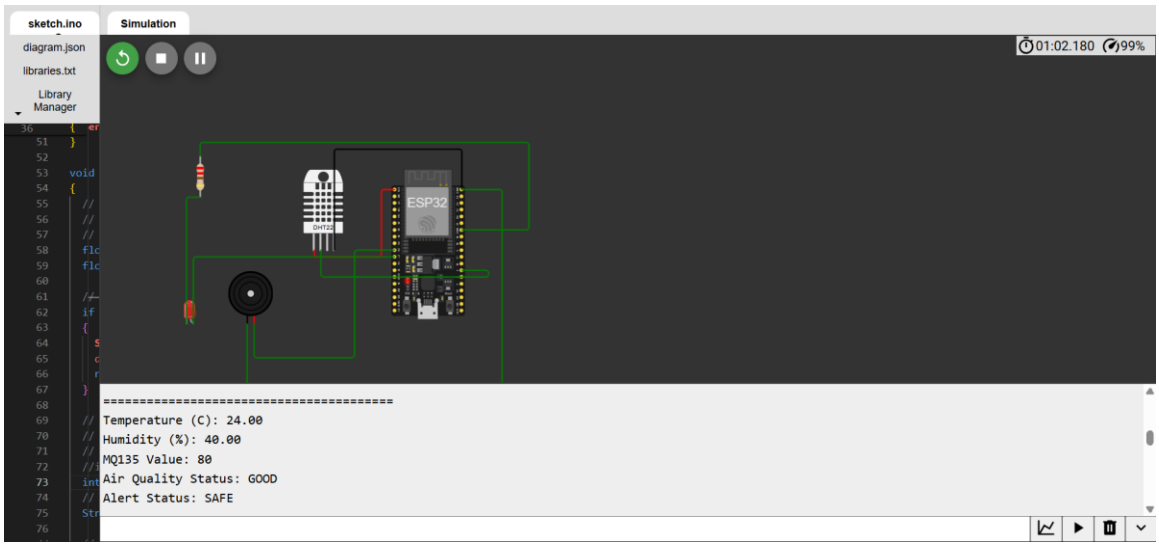

Figure 4: System Output Under Good Air Quality Conditions

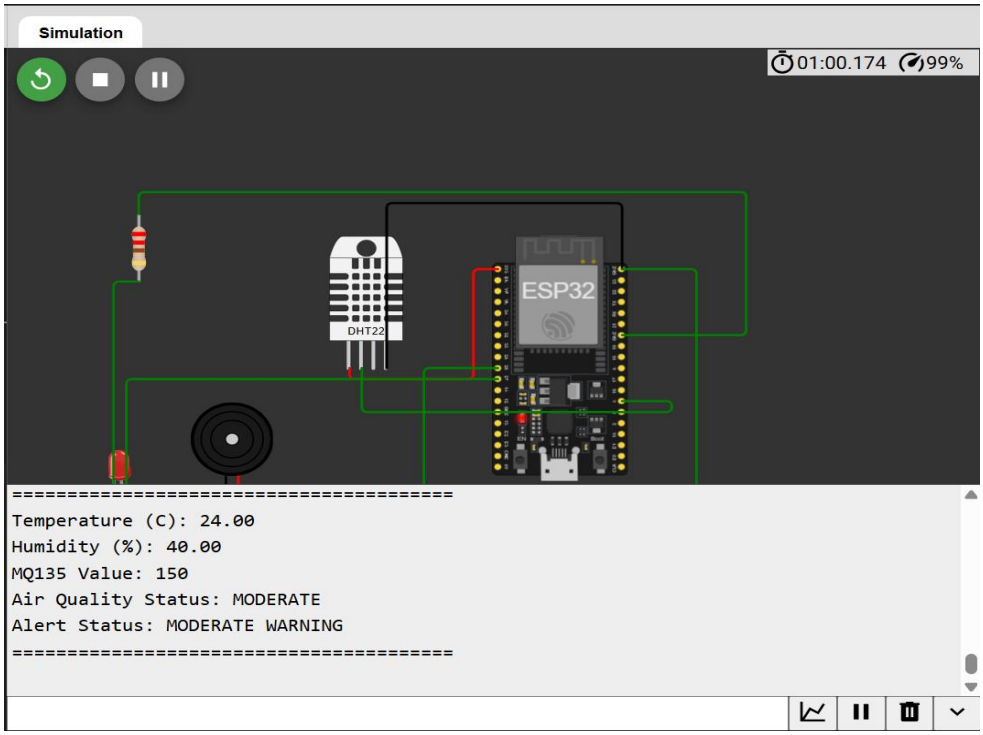

Figure 5: System Output Under Moderate Air Quality Conditions

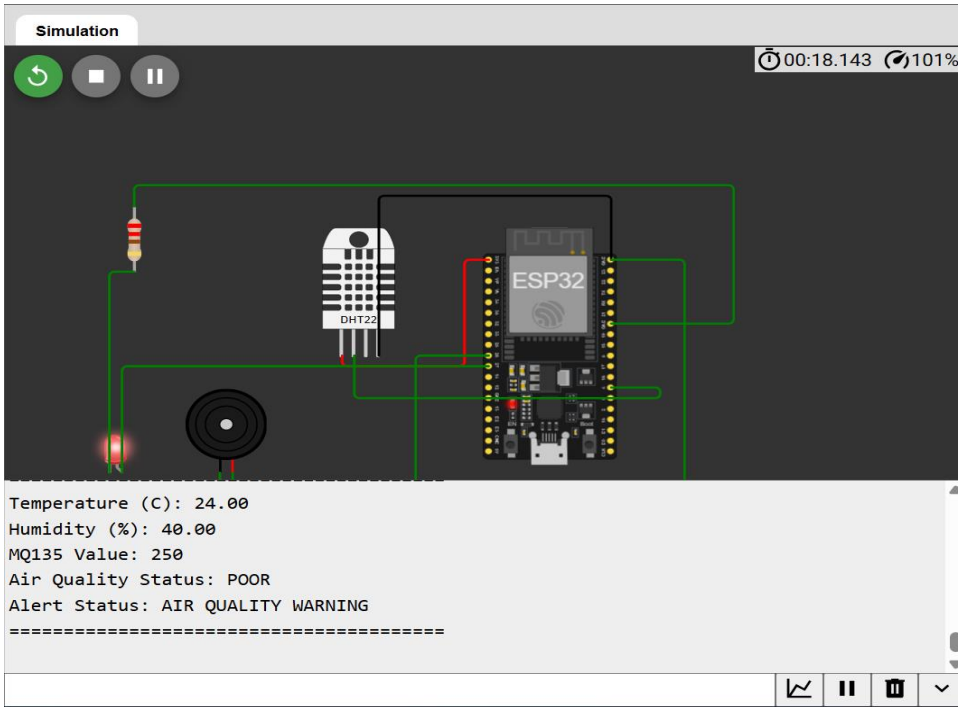

Figure 6: System Output Under Poor Air Quality Conditions

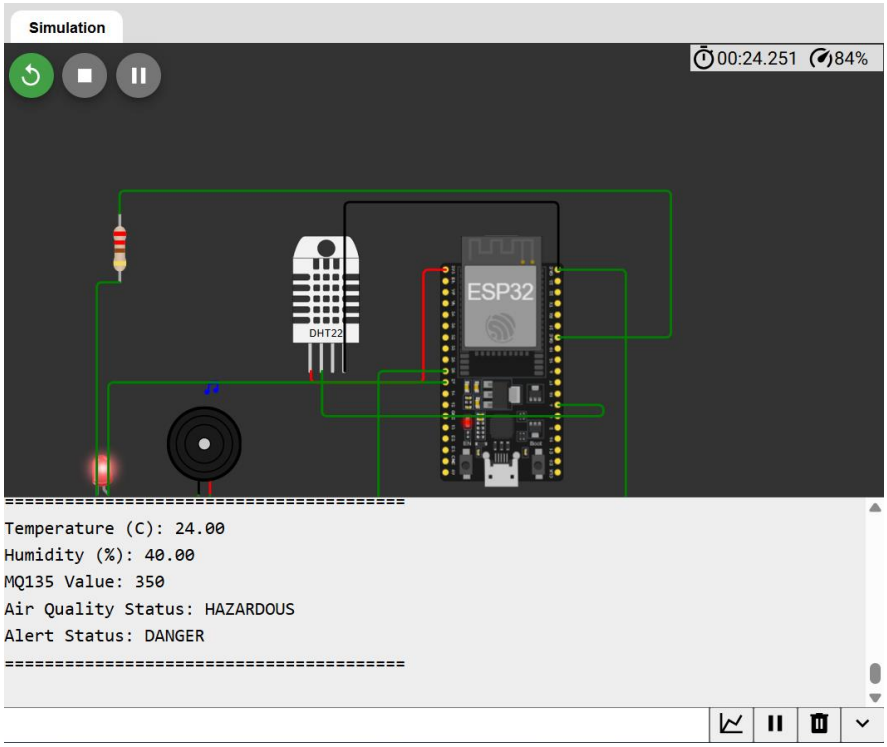

Figure 7: System Output Under Hazardous Air Quality Conditions

Dataset Description

The dataset contains timestamps, air-quality values, temperature, humidity, pollution-status codes, and alert-status codes. The exported CSV can support future analytics, machine learning experiments, and trend analysis.

GitHub Repository

Repository: IoT-Air-Quality-Monitoring-Dashboard

The repository follows professional software engineering practices with dedicated folders for source code, datasets, screenshots, architecture diagrams, and documentation.

Applications

Smart City Monitoring
Industrial Safety Monitoring
Environmental Research
Indoor Air Quality Monitoring
Educational IoT Laboratories
Building Automation

Advantages

Low Cost
Cloud Connected
Scalable Design
Historical Data Storage
Remote Monitoring
Industry-Relevant Skills

Limitations

The MQ135 sensor values are simulated. A real-world deployment would require calibration, field testing, and sensor validation.

Future Enhancements

Machine Learning-Based Prediction
Mobile Application Integration
Email Alerts
SMS Notifications
Multi-Node Monitoring
GIS Mapping
Edge AI Processing

Placement Relevance

This project demonstrates practical knowledge of ESP32 programming, cloud integration, APIs, data logging, dashboard development, GitHub documentation, IoT architecture, and system design. These skills are directly relevant to software engineering, IoT engineering, cloud engineering, and data analytics roles.

Interview Discussion Points

Candidates can explain sensor integration, cloud communication, ThingSpeak APIs, data flow architecture, alert systems, dashboard design, CSV analytics, scalability considerations, and future improvements.

Conclusion

The project successfully implements an end-to-end IoT monitoring platform capable of sensing, processing, transmitting, visualizing, and analyzing environmental data. The solution provides a strong foundation for advanced IoT, cloud, and analytics applications.

References

ESP32 Documentation
Arduino Documentation
ThingSpeak Documentation
DHT22 Documentation
Wokwi Documentation
Research Articles on IoT Environmental Monitoring